

Dakar Institute of Technology



Habilitation n°3386-21 FEV.2024

Domaine : Sciences et Technologies

Département : Informatique

Spécialité : Intelligence Artificielle

Master 2 IA : Examen de Technologie BigData

Projet : Système Big Data d'analyse
des ventes de livres

Présenté par

Djidjooh Mathieu Maurice AHOUANSOU

Fatoumata Bintou SIDIBE

Abdin KOUSSUBE

Année Académique : 2025 -2026

SOMMAIRE

LISTE DES FIGURES	iii
RÉSUMÉ	iv
1 Généralités	1
1.1 Introduction	1
1.2 Problématique	1
1.3 Objectifs du projet	2
1.4 Structure du projet	3
1.5 Clarification des concepts et Cadre théorique	3
1.5.1 Clarification des concepts	3
1.5.2 Architecture globale du projet BookFlow	5
2 Approche méthodologique	9
2.1 Phase de cadrage et analyse des besoins	9
2.2 Conception de l'architecture globale	9
2.3 Mise en place de la base source	10
2.4 Mise en place du Data Warehouse Cloud	10
2.5 Développement du pipeline d'ingestion	10
2.6 Transformation des données avec DBT	10
2.6.1 RAW vers STAGGING	11
2.6.2 STAGGING vers WAREHOUSE	11
2.6.3 WAREHOUSE vers MARTS	11
2.7 Automatisation avec Airflow	11
2.8 Développement du dashboard décisionnel	11
2.9 Tests, validation et optimisation	12
2.10 Documentation et restitution	12
3 Spécification des besoins et conception	14
3.1 Etude des besoins	14
3.1.1 Les besoins fonctionnels	14
3.1.2 Les besoins non fonctionnels	15
3.1.3 Acteurs du projet	15

3.2	Conception du projet	15
3.2.1	Choix technologiques	15
3.2.2	Architecture logique	16
3.2.3	Modèle de données	16
3.2.4	Flux de traitement	17
3.3	Contraintes du projet	17
3.4	Résultats attendus	17
3.5	Conclusion	17
4	Réalisation du projet	18
4.1	Environnement de travail	18
4.1.1	Environnement matériel	18
4.1.2	Environnement logiciel et plateforme cloud	18
4.2	Processus de la réalisation	20
4.2.1	Création des repectoires et l'architecture du projet	21
4.2.2	Création de la base locale PostgreSQL	24
4.2.3	Ingestion vers Snowflake	25
4.2.4	Transformation RAW vers STAGGING	27
4.2.5	Transformation STAGGING vers WAREHOUSE	28
4.2.6	Création du MART OBT Sales	30
4.2.7	Orchestration avec Airflow	32
4.2.8	visualisation avec Streamlit	34
	Conclusion	38

LISTE DES FIGURES

Table des figures

1.1	Architecture du projet BookFlow	6
2.1	Flowchart de la Méthologie du Projet Bookflow	13
4.1	Commandes de créations des différents dossiers du projet	21
4.2	Verification des dossiers créés	22
4.3	Screenshot de la création du fichier docker-compose.yml	23
4.4	Screenshot montrant la création des conteneurs nécessaires du projet	23
4.5	Création des tables	24
4.6	Vérification des données	24
4.7	Flowchart du processus d'ingestion des données dans Snowflake	26
4.8	Script de creation des tables et ingestion dans Snowflake	26
4.9	Verification de la presence des donnee dans Snowflake	27
4.10	Screenshot de la transformation RAW vers STAGGING	28
4.11	Screenshot de la transformation STAGGING vers WAREHOUSE	30
4.12	Screenshot de Création du MART OBT Sales	32
4.13	Flowchart de l'ingestion vers Snowflake et de l'orchestration avec Airflow	33
4.14	DAG d'orchestration avec Airflow	33
4.15	Interface Airflow montrant la réussite de l'orchestration	34
4.16	Dashboard montrant les ventes globales	35
4.17	Dashboard montrant les ventes de livres par année	36
4.18	Dashboard montrant les Top livres vendus	36
4.19	Dashboard montrant les meilleurs clients	37

RÉSUMÉ

Le présent projet, intitulé BookFlow Analytics Platform, porte sur la conception et la mise en œuvre d'une architecture moderne de traitement et de valorisation des données orientée décisionnel. L'objectif principal était de développer une chaîne complète de collecte, de transformation, de stockage et de visualisation des données commerciales d'une librairie, en mobilisant des outils couramment utilisés dans le domaine du Data Engineering.

L'architecture développée repose sur plusieurs technologies complémentaires. Les données transactionnelles sont initialement stockées dans une base PostgreSQL, utilisée comme système source. Elles sont ensuite ingérées vers l'entrepôt de données cloud Snowflake, qui constitue la plateforme analytique centrale du projet. Les transformations ont été réalisées à l'aide de dbt (Data Build Tool) selon une approche en couches comprenant les schémas RAW, STAGGING, WAREHOUSE et MARTS. Cette structuration permet d'assurer la qualité, la traçabilité et l'organisation des données.

L'automatisation du pipeline a été assurée avec Apache Airflow, à travers la création d'un DAG orchestrant successivement l'ingestion des données, les transformations analytiques et la production de la table finale OBT_SALES. Cette table consolidée regroupe les principales informations nécessaires aux analyses commerciales.

Enfin, un tableau de bord interactif a été développé avec Streamlit afin de restituer les indicateurs clés tels que le chiffre d'affaires, les volumes de ventes, les tendances annuelles et les performances clients. Cette interface permet une exploitation simple et intuitive des résultats obtenus.

Ce projet démontre la pertinence des architectures modernes de données pour transformer des données opérationnelles en informations stratégiques utiles à la prise de décision. Il constitue également une application concrète des compétences en Big Data, Cloud Computing, automatisation et Business Intelligence.

Chapitre 1: Généralités

1.1 Introduction

À l'ère du numérique, les entreprises génèrent quotidiennement un volume croissant de données provenant de leurs activités commerciales, de leurs clients et de leurs opérations internes. Dans le secteur de la vente de livres, chaque transaction, facture, commande ou interaction client représente une source précieuse d'informations pouvant être exploitée pour améliorer la prise de décision stratégique. Toutefois, ces données sont souvent dispersées dans plusieurs systèmes et sous différents formats, rendant leur exploitation complexe sans une architecture adaptée.

Face à cette réalité, les technologies Big Data et Cloud offrent aujourd'hui des solutions performantes pour collecter, stocker, transformer et analyser efficacement ces données à grande échelle. Les plateformes modernes de Data Warehouse, associées à des outils d'orchestration et de transformation, permettent de structurer les flux de données et de produire des indicateurs fiables en temps réel [Kimball and Ross \(2013\)](#).

C'est dans ce contexte que s'inscrit le projet BookFlow, dont l'objectif est de concevoir une chaîne complète de traitement de données appliquée à un système de vente de livres. Le projet repose sur l'utilisation de technologies modernes telles que Snowflake pour le stockage analytique, DBT pour les transformations de données, Airflow pour l'automatisation des traitements, ainsi qu'un outil de visualisation pour la restitution des résultats.

À travers cette réalisation, il s'agit non seulement de mettre en pratique les concepts fondamentaux de l'architecture Big Data, mais également de démontrer comment la donnée peut devenir un véritable levier de performance pour une entreprise commerciale.

1.2 Problématique

Dans un environnement commercial de plus en plus concurrentiel, les entreprises spécialisées dans la vente de livres doivent disposer d'informations fiables, rapides et exploitables afin d'optimiser leurs performances. Les décisions liées à la gestion des stocks, au suivi des ventes, à l'identification des livres les plus demandés, au comportement des clients ou encore à la rentabilité des activités reposent désormais sur une exploitation efficace des

données. Pourtant, dans de nombreuses structures, ces informations restent fragmentées entre plusieurs bases de données opérationnelles, fichiers Excel ou applications métiers, ce qui limite considérablement leur valorisation.

Le principal défi réside dans la difficulté à transformer ces données brutes en indicateurs stratégiques utilisables par les décideurs. Les systèmes transactionnels classiques sont conçus pour enregistrer les opérations quotidiennes, mais ils ne sont pas toujours adaptés aux besoins d'analyse avancée, de reporting dynamique ou de pilotage en temps réel. Cette situation engendre souvent des retards dans la production des rapports, des incohérences dans les chiffres, une faible visibilité sur la performance globale de l'entreprise et des prises de décision basées sur des informations incomplètes.

Dans le cas d'un système de vente de livres, plusieurs questions essentielles peuvent se poser : quels sont les ouvrages les plus vendus par période ? Quelles catégories génèrent le plus de chiffre d'affaires ? Quels clients réalisent les achats les plus importants ? Quelle est l'évolution mensuelle des ventes ? Quels sont les écarts entre les montants facturés et les paiements reçus ? Sans une architecture de données performante, répondre rapidement et précisément à ces interrogations devient complexe.

Par ailleurs, l'augmentation continue du volume des transactions impose l'adoption de solutions évolutives, capables de traiter de grandes quantités de données tout en garantissant la rapidité d'exécution, la sécurité et la disponibilité des informations. Les approches traditionnelles montrent rapidement leurs limites face aux exigences actuelles de scalabilité, d'automatisation et d'analyse multidimensionnelle.

Dès lors, la problématique centrale de ce projet peut être formulée ainsi : comment concevoir une architecture Big Data moderne permettant de collecter, centraliser, transformer et analyser efficacement les données d'un système de vente de livres, afin de produire des indicateurs fiables et d'améliorer la prise de décision ?

Pour répondre à cette problématique, il apparaît nécessaire de mettre en place une solution intégrant un Data Warehouse cloud, des processus automatisés de transformation de données, une orchestration intelligente des traitements ainsi qu'un tableau de bord interactif facilitant l'analyse décisionnelle.

1.3 Objectifs du projet

L'objectif général de ce projet vise est de développer une architecture Big Data performante permettant de collecter, transformer, centraliser et analyser les données d'un système de vente de livres afin d'améliorer la prise de décision stratégique.

Spécifiquement il s'agit de :

1. mettre en place un pipeline d'ingestion et de stockage des données issues du système de vente dans un environnement Data Warehouse structuré ;

2. développer des processus automatisés de transformation des données pour produire des tables analytiques fiables et exploitables ;
3. créer un tableau de bord interactif permettant de visualiser les indicateurs clés de performance des ventes, des clients et des produits.

1.4 Structure du projet

Le présent travail est structuré en plusieurs chapitres complémentaires. Le **Chapitre 1** est consacré aux généralités et présente le contexte, la problématique, les objectifs ainsi que les principales notions théoriques liées au traitement automatique du langage naturel et à l'analyse des sentiments. Le **Chapitre 2** décrit en détail la méthodologie adoptée, notamment les étapes de collecte et de préparation des données, les techniques de vectorisation, ainsi que les modèles de classification classiques et hybrides retenus. Le **Chapitre 3** expose les résultats obtenus, accompagnés d'analyses critiques et de visualisations avancées permettant d'interpréter les performances des différents modèles. Enfin, le **Chapitre 4** propose une discussion générale, souligne les limites rencontrées et formule des perspectives d'amélioration et de recherche future.

1.5 Clarification des concepts et Cadre théorique

1.5.1 Clarification des concepts

Afin de mieux comprendre le projet BookFlow, il est important de préciser certains concepts clés liés à l'architecture Big Data et à la gestion des données décisionnelles.

1.5.1.1 Big Data

Le Big Data désigne l'ensemble des technologies et méthodes permettant de collecter, stocker, traiter et analyser de très grands volumes de données provenant de sources variées et générées à grande vitesse. Dans ce projet, les données concernent les ventes, les factures, les livres, les clients et les catégories de produits.

1.5.1.2 Base de données transactionnelle (OLTP)

Une base de données OLTP (Online Transaction Processing) est conçue pour gérer les opérations quotidiennes telles que les ventes, les paiements ou les enregistrements clients. Elle privilégie la rapidité des insertions et mises à jour. PostgreSQL ou MySQL peuvent jouer ce rôle dans ce projet [Damji et al. \(2020\)](#).

1.5.1.3 Data Warehouse

Le Data Warehouse est un entrepôt de données centralisé destiné à l'analyse et au reporting. Contrairement aux bases transactionnelles, il est optimisé pour les requêtes analytiques complexes. Dans ce projet, Snowflake sert de Data Warehouse.

1.5.1.4 Schéma en couches de données

Le projet repose sur une organisation des données en plusieurs couches :

- ✓ **RAW** : contient les données brutes importées depuis la base source, sans modification.
- ✓ **STAGGING** : zone de préparation et de nettoyage des données.
- ✓ **WAREHOUSE** : données modélisées sous forme de dimensions et faits.
- ✓ **MARTS** : tables finales orientées métier pour l'analyse et la visualisation.

1.5.1.5 ETL / ELT

- ✓ **ETL** : Extract, Transform, Load (extraire, transformer puis charger).
- ✓ **ELT** : Extract, Load, Transform (extraire, charger puis transformer).

Dans ce projet, l'approche utilisée est principalement ELT, car les données sont d'abord chargées dans Snowflake puis transformées avec DBT.

1.5.1.6 DBT (Data Build Tool)

DBT est un outil moderne de transformation de données permettant d'écrire des modèles SQL pour nettoyer, structurer et documenter les données dans le Data Warehouse. Il facilite l'automatisation et la traçabilité des traitements.

1.5.1.7 Airflow

Apache Airflow est un outil d'orchestration permettant de planifier, automatiser et surveiller les différentes tâches du pipeline de données. Il permet par exemple de lancer automatiquement les transformations DBT chaque jour [Apache Software Foundation \(2024\)](#).

1.5.1.8 Table de faits et dimensions

Dans un modèle décisionnel :

- ✓ **Table de faits** : contient les mesures quantitatives (quantité vendue, montant total, prix unitaire).
- ✓ **Tables de dimensions** : contiennent les informations descriptives (clients, livres, catégories, dates) [Kimball and Ross \(2013\)](#).

1.5.1.9 OBT (One Big Table))

Une One Big Table est une table unique regroupant toutes les informations nécessaires à l'analyse métier. Dans ce projet, MARTS.obt_sales centralise les données des ventes, clients, factures et livres pour simplifier les analyses [dbt Labs \(2024\)](#).

1.5.1.10 Dashboard décisionnel

Un dashboard est une interface visuelle permettant de suivre les indicateurs clés : chiffre d'affaires, ventes mensuelles, livres les plus vendus, catégories les plus rentables, etc. Il facilite la prise de décision rapide [Streamlit Inc. \(2024\)](#).

1.5.1.11 KPI (Key Performance Indicators)

Les KPI sont des indicateurs de performance utilisés pour mesurer les résultats de l'entreprise. Exemples :

- ✓ Chiffre d'affaires total
- ✓ Quantité vendue
- ✓ Taux de paiement
- ✓ Top clients
- ✓ Top livres vendus

1.5.1.12 Cloud Computing

Le Cloud Computing permet d'utiliser des ressources informatiques à distance via Internet. Snowflake fonctionne dans le cloud, ce qui offre flexibilité, performance et évolutivité sans gérer directement les serveurs [Kleppmann \(2017\)](#).

1.5.2 Architecture globale du projet BookFlow

L'architecture globale du projet BookFlow repose sur une chaîne complète de traitement de données allant de la collecte des données transactionnelles jusqu'à leur visualisation sous forme d'indicateurs décisionnels. Elle suit une logique ELT : les données sont d'abord extraites et chargées dans Snowflake, puis transformées progressivement avec DBT. La Figure 1.1 présente le schéma conceptuel de l'architecture global proposé pour ce projet.

1.5.2.1 Couche source : base de données locale

La première couche correspond au système opérationnel de vente de livres. Cette base est créée avec PostgreSQL. Elle contient les tables de base du système commercial :

- ✓ customers

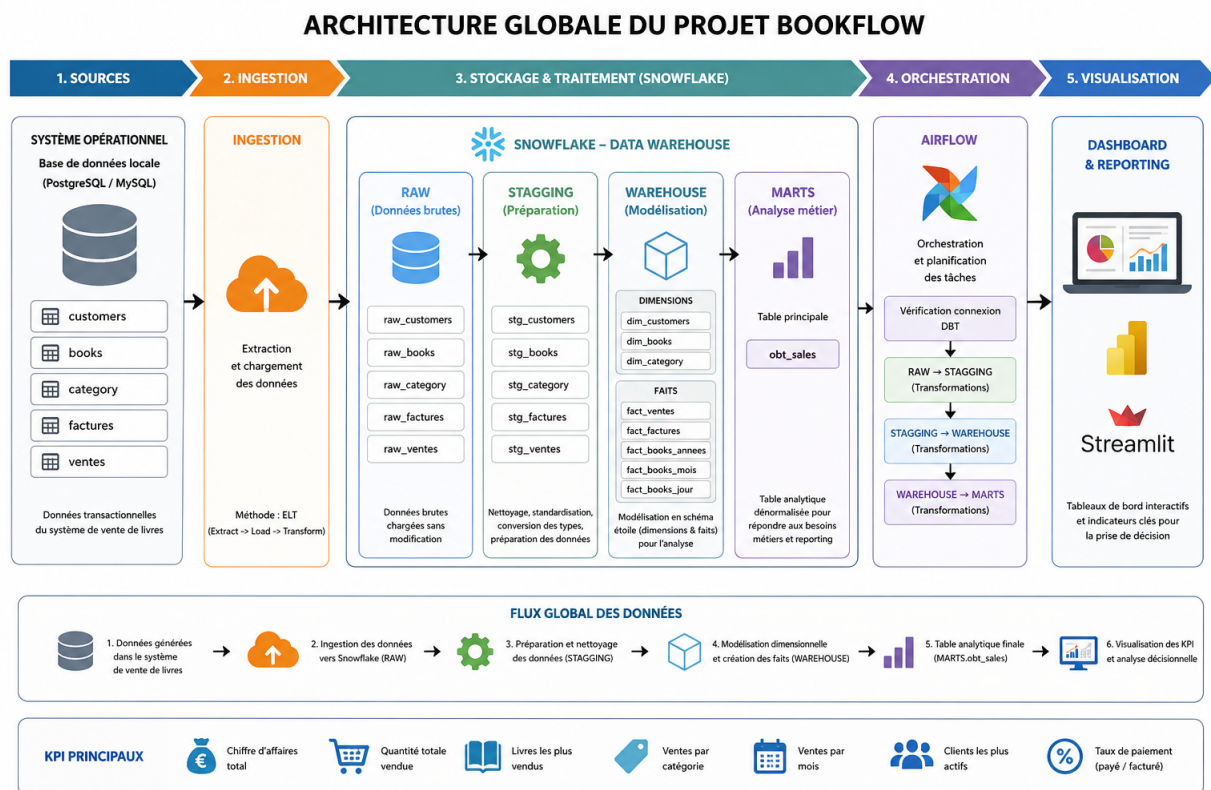


FIGURE 1.1 – Architecture du projet BookFlow

- ✓ Books
- ✓ Category
- ✓ Ventes
- ✓ Factures

1.5.2.2 Couche d'ingestion

La couche d'ingestion assure le transfert des données depuis la base locale vers Snowflake. Son rôle est de déposer les données brutes dans le schéma RAW sans transformation majeure. Cette étape permet de conserver une copie fidèle des données sources dans le Data Warehouse.

1.5.2.3 LCouche RAW dans Snowflake

Le schéma RAW contient les données brutes issues du système source. Les tables y sont stockées dans leur état initial.Exemple :

- ✓ RAW.customers
- ✓ RAW.Books
- ✓ RAW.Category
- ✓ RAW.Ventes
- ✓ RAW.Factures

1.5.2.4 Couche STAGGING

La couche STAGGING correspond à la préparation des données. Elle permet de nettoyer, convertir et standardiser les données brutes. Dans ce projet, les principales transformations sont :

- ✓ **Conversion des dates YYYYMMDD en format DATE**
- ✓ **Renommage ou standardisation des colonnes**
- ✓ **Copie propre des tables sources**
- ✓ **Préparation des données pour le modèle décisionnel**

Exemple :

- ✓ **STAGGING.stg_customers**
- ✓ **STAGGING.stg_Books**
- ✓ **STAGGING.stg_Category**
- ✓ **STAGGING.stg_Ventes**
- ✓ **STAGGING.stg_Factures**

1.5.2.5 Couche WAREHOUSE

La couche WAREHOUSE représente le cœur analytique du projet. Les données y sont organisées sous forme de tables de dimensions et de tables de faits. Les dimensions décrivent les entités métier :

- ✓ **dim_customers**
- ✓ **dim_Books**
- ✓ **dim_Category**

Les tables de faits contiennent les mesures commerciales :

- ✓ **fact_Ventes**
- ✓ **fact_Factures**
- ✓ **fact_Books_annees**
- ✓ **fact_Books_mois**
- ✓ **fact_Books_jour**

Cette couche permet d'analyser les ventes selon plusieurs axes : client, livre, catégorie, année, mois et jour.

1.5.2.6 Couche MARTS

La couche MARTS contient les tables finales destinées directement à l'analyse métier et à la visualisation. La table principale est : **MARTS.obt_sales**. Elle regroupe les informations essentielles provenant des ventes, factures, livres, catégories et clients. Elle permet de répondre rapidement à des questions comme :

- ✓ **Quels livres sont les plus vendus ?**
- ✓ **Quel est le chiffre d'affaires par mois ?**

- ✓ **Quels clients achètent le plus ?**
- ✓ **Quelles catégories génèrent le plus de revenus ?**
- ✓ **Quel est le montant payé par rapport au montant facturé ?**

1.5.2.7 Couche d'orchestration avec Airflow

Airflow permet d'automatiser l'exécution des traitements. Il organise les tâches sous forme de DAG. Grâce à Airflow, le pipeline peut être lancé automatiquement selon une fréquence définie.

1.5.2.8 Couche de visualisation

La dernière couche est consacrée à la restitution des résultats. Elle peut être réalisée avec Streamlit, Power BI ou un notebook. Le dashboard permet de présenter les KPI principaux :

- ✓ **Chiffre d'affaires total**
- ✓ **Quantité totale vendue**
- ✓ **Livres les plus vendus**
- ✓ **Ventes par catégorie**
- ✓ **Ventes par mois**
- ✓ **Clients les plus actifs**
- ✓ **Taux de paiement**

En résumé, l'architecture BookFlow est construite autour d'un Data Warehouse cloud organisé en plusieurs couches. Les données issues d'une base transactionnelle locale sont ingérées dans Snowflake dans une zone brute appelée RAW. Elles sont ensuite préparées dans la couche STAGGING, puis modélisées dans la couche WAREHOUSE sous forme de dimensions et de faits. Enfin, une table analytique finale MARTS.obt_sales est créée pour faciliter les analyses décisionnelles. L'ensemble du processus est automatisé par Airflow et valorisé à travers un tableau de bord interactif.

Chapitre 2: Approche méthodologique

La réalisation du projet BookFlow s'appuie sur une démarche structurée, progressive et orientée résultats, permettant de concevoir une chaîne complète de traitement de données depuis la source transactionnelle jusqu'à la restitution décisionnelle. L'approche adoptée combine les principes de l'ingénierie des données, du Data Warehousing moderne et de l'automatisation des pipelines Big Data.

2.1 Phase de cadrage et analyse des besoins

Cette première étape a consisté à la compréhension du contexte métier du système de vente de livres et à l'identification des besoins décisionnels de la structure. Il s'est agi de définir les principales problématiques à résoudre ainsi que les indicateurs attendus. Les actions menées sont :

- ✓ analyse de l'énoncé du projet ;
- ✓ identification des sources de données disponibles ;
- ✓ compréhension des relations entre les tables (clients, livres, ventes, factures, catégories) ;
- ✓ définition des KPI à produire (chiffre d'affaires, quantités vendues, top livres, ventes mensuelles, clients actifs, etc.) ;
- ✓ choix des technologies adaptées : PostgreSQL, Snowflake, DBT, Airflow et outil de visualisation.

Cette phase a permis de poser les bases techniques et fonctionnelles du projet.

2.2 Conception de l'architecture globale

Après l'analyse des besoins, une architecture cible est définie afin d'organiser le flux des données de manière cohérente. Le projet repose sur une architecture en couches : Source → RAW → STAGGING → WAREHOUSE → MARTS → Dashboard. Chaque couche joue un rôle spécifique :

- ✓ **Source** : base transactionnelle locale ;
- ✓ **RAW** : données brutes ;

- ✓ **STAGGING** : préparation des données ;
- ✓ **WAREHOUSE** : modélisation décisionnelle ;
- ✓ **MARTS** : tables analytiques finales ;
- ✓ **Dashboard** : visualisation.

Cette architecture favorise la traçabilité, la qualité et l'évolutivité des traitements.

2.3 Mise en place de la base source

Une base de données locale MySQL a été créée afin de simuler le système opérationnel de vente de livres. Les tables métiers y sont implémentées conformément au modèle fourni :

- ✓ **customers**
- ✓ **Books**
- ✓ **Category**
- ✓ **Ventes**
- ✓ **Factures**

Des données de test réalistes sont ensuite insérées afin d'alimenter les traitements futurs.

2.4 Mise en place du Data Warehouse Cloud

Un environnement Snowflake est configuré pour accueillir les données analytiques. La base BOOKSHOP est créée avec quatre schémas :

- ✓ **RAW**
- ✓ **STAGGING**
- ✓ **WAREHOUSE**
- ✓ **VMARTS**

Cette organisation permet une séparation claire entre données brutes, données préparées et données décisionnelles.

2.5 Développement du pipeline d'ingestion

Un mécanisme d'ingestion est mis en place pour transférer les données depuis la base locale vers Snowflake. Cette étape peut être réalisée via un scripts Python. L'objectif est de charger les tables sources dans le schéma RAW sans modification structurelle.

2.6 Transformation des données avec DBT

Les transformations sont développées avec DBT selon une logique modulaire :

2.6.1 RAW vers STAGGING

Nettoyage et standardisation :

- ✓ conversion des dates YYYYMMDD en type DATE ;
- ✓ uniformisation des noms de colonnes ;
- ✓ contrôle de cohérence ;
- ✓ préparation des tables stg_*

2.6.2 STAGGING vers WAREHOUSE

Création du modèle décisionnel :

- ✓ **tables de dimensions** : dim_customers, dim_books, dim_category ; ;
- ✓ **tables de faits** : fact_ventes, fact_factures ;
- ✓ **agrégations temporelles par année, mois et jour.**

2.6.3 WAREHOUSE vers MARTS

Création de la table finale : MARTS.obt_sales. Cette table centralise toutes les informations utiles à l'analyse métier.

2.7 Automatisation avec Airflow

L'exécution des traitements est orchestrée avec Apache Airflow. Un DAG est développé afin d'automatiser les étapes suivantes : Ingestion → DBT STAGGING → DBT WAREHOUSE → DBT MARTS. Cette automatisation garantit :

- ✓ exécution planifiée ;
- ✓ suivi des tâches ;
- ✓ reprise en cas d'erreur ;
- ✓ industrialisation du pipeline.

2.8 Développement du dashboard décisionnel

Un tableau de bord interactif est conçu avec Streamlit ou Power BI afin de valoriser les données finales. Les visualisations proposées incluent :

- ✓ chiffre d'affaires global ;
- ✓ ventes par mois ;
- ✓ top livres vendus ;
- ✓ ventes par catégorie ;
- ✓ meilleurs clients ;
- ✓ taux de paiement.

Cette phase a permis de transformer les données en outil d'aide à la décision.

2.9 Tests, validation et optimisation

Avant la finalisation, plusieurs vérifications ont été réalisées :

- ✓ conformité des données ;
- ✓ cohérence entre source et Snowflake ;
- ✓ performance des requêtes ;
- ✓ exactitude des indicateurs ;
- ✓ robustesse du pipeline Airflow.

Des optimisations peuvent ensuite être apportées pour améliorer les temps de traitement.

2.10 Documentation et restitution

Enfin, l'ensemble du travail est documenté dans :

- ✓ un rapport technique détaillé ;
- ✓ une présentation PowerPoint ;
- ✓ une démonstration vidéo du pipeline.

L'approche méthodologique adoptée repose sur une logique incrémentale allant de la donnée brute à la valeur métier. Elle combine ingestion, transformation, modélisation, automatisation et visualisation dans une architecture Big Data moderne, scalable et orientée décisionnel. La Figure [2.1](#) présente la synthèse de l'approche méthodologique utilisée.

.

MÉTHODOLOGIE DU PROJET BOOKFLOW – FLOWCHART



FIGURE 2.1 – Flowchart de la Méthologie du Projet Bookflow

Chapitre 3: Spécification des besoins et conception

La réussite d'un projet Big Data repose en grande partie sur une bonne compréhension des besoins métier et sur une conception technique cohérente. Avant tout développement, il est essentiel d'identifier les attentes des utilisateurs, les contraintes techniques ainsi que les résultats attendus. Cette étape permet de construire une solution adaptée, évolutive et performante. Dans le cadre du projet BookFlow, la phase de spécification des besoins et de conception vise à définir clairement les fonctionnalités attendues, les données à exploiter, les indicateurs à produire ainsi que l'architecture technique permettant de répondre efficacement aux objectifs du projet. Ce chapitre présente les spécifications de la solution développée. Il comprend deux parties distinctes à savoir :

- ✓ les besoins identifiés, qu'ils soient fonctionnels ou non fonctionnels ;
- ✓ la conception où une représentation des interactions utilisateur pour mieux comprendre le fonctionnement global de l'application.

3.1 Etude des besoins

La définition des besoins constitue une étape cruciale dans la conception et la réalisation d'un projet. Elle a pour objectif d'identifier les fonctionnalités essentielles du système ainsi que les contraintes et critères de performance indispensables à son bon fonctionnement. Cette phase permet donc de préciser les exigences spécifiques du projet, qu'elles soient fonctionnelles ou non fonctionnelles.

3.1.1 Les besoins fonctionnels

Les besoins fonctionnels correspondent aux fonctionnalités essentielles que l'application doit garantir afin d'atteindre les objectifs fixés du projet. Dans le cadre de ce projet Bookflow, Le système doit permettre de :

- ✓ collecter les données provenant de la base transactionnelle (clients, livres, ventes, factures, catégories) ;
- ✓ centraliser les données dans un Data Warehouse cloud ;

- ✓ transformer les données brutes en données propres et exploitables ;
- ✓ construire des tables analytiques facilitant les requêtes métier ;
- ✓ produire une table consolidée des ventes (obt_sales) ;
- ✓ générer des indicateurs clés de performance (KPI) ;
- ✓ visualiser les résultats à travers un tableau de bord interactif ;
- ✓ automatiser l'exécution des traitements de données.

3.1.2 Les besoins non fonctionnels

Les besoins non fonctionnels concernent la qualité globale du système. La solution doit garantir :

- ✓ **Performance** : temps de traitement rapide ;
- ✓ **Fiabilité** : exactitude et cohérence des données ;
- ✓ **Scalabilité** : capacité à gérer un volume croissant de données ;
- ✓ **Sécurité** : protection des accès aux données ;
- ✓ **Disponibilité** : accès permanent au système ;
- ✓ **Maintenabilité** : facilité d'évolution du pipeline ;
- ✓ **Traçabilité** : Traçabilité : suivi des traitements exécutés.

3.1.3 Acteurs du projet

Les principaux utilisateurs de la solution sont :

- ✓ **Direction commerciale** : suivi des ventes et du chiffre d'affaires ;
- ✓ **Responsable marketing** : analyse des tendances d'achat ;
- ✓ **Gestionnaire stock** : identification des livres les plus vendus ;
- ✓ **Administrateur technique** : supervision du pipeline de données ;
- ✓ **Analyste métier** : exploitation des tableaux de bord.

3.2 Conception du projet

La conception vise à établir la structure et le fonctionnement global de la solution. Elle décrit la manière dont les différents acteurs, y compris le système, interagissent pour atteindre les objectifs du projet. Cette étape permet de modéliser les fonctionnalités essentielles et de clarifier les processus afin d'assurer une vision cohérente du projet.

3.2.1 Choix technologiques

Pour répondre efficacement aux besoins, les outils suivants ont été retenus :

TABLE 3.1 – Technologies utilisées dans le projet

Composant	Outil choisi	Rôle
Base source	PostgreSQL	Stockage transactionnel
Data Warehouse	Snowflake	Stockage analytique cloud
Transformation	DBT	Modélisation et transformations SQL
Orchestration	Airflow	Automatisation des pipelines
Visualisation	Streamlit	Dashboard décisionnel

3.2.2 Architecture logique

Le projet adopte une architecture en couches :

Source → RAW → STAGGING → WAREHOUSE → MARTS → Dashboard

Description des couches :

- **Source** : base opérationnelle locale ;
- **RAW** : données brutes importées ;
- **STAGGING** : nettoyage et standardisation ;
- **WAREHOUSE** : dimensions et faits ;
- **MARTS** : table finale OBT ;
- **Dashboard** : restitution visuelle.

3.2.3 Modèle de données

Les principales tables utilisées sont :

Tables sources

- customers
- books
- category
- factures
- ventes

Tables décisionnelles

- dim_customers
- dim_books
- dim_category
- fact_ventes
- fact_factures
- obt_sales

3.2.4 Flux de traitement

Le traitement des données suit les étapes suivantes :

Extraction → RAW → DBT → WAREHOUSE → MARTS → Dashboard

3.3 Contraintes du projet

Certaines contraintes ont été prises en compte :

- Temps limité de réalisation ;
- Utilisation de versions gratuites ou open source ;
- Ressources matérielles limitées ;
- Besoin d'automatisation complète ;
- Respect du cahier des charges académique.

3.4 Résultats attendus

À la fin du projet, la solution devra permettre :

- Une centralisation fiable des données ;
- Une analyse rapide des ventes ;
- Des tableaux de bord dynamiques ;
- Une automatisation complète du pipeline ;
- Une architecture Big Data moderne et professionnelle.

3.5 Conclusion

La phase de spécification des besoins et de conception constitue une étape déterminante dans le projet **BookFlow**. Elle permet d'aligner les besoins métier avec les solutions technologiques retenues. Grâce à cette démarche structurée, le projet dispose d'une base solide pour la mise en œuvre d'un système décisionnel performant, automatisé et orienté Business Intelligence.

Chapitre 4: Réalisation du projet

Ce chapitre présente tout d'abord l'environnement de travail, en décrivant les outils matériels et logiciels mobilisés. Il expose ensuite les différentes étapes de mise en œuvre du projet d'analyse des sentiments. Enfin, il propose une brève présentation de l'interface graphique développée afin de rendre l'application plus intuitive et accessible aux utilisateurs.

4.1 Environnement de travail

Dans cette section, il est présenté les outils et ressources mobilisés pour la mise en œuvre du projet. L'environnement de travail est structuré en deux volets principaux : l'environnement matériel et l'environnement logiciel.

4.1.1 Environnement matériel

Pour garantir la rapidité et les performances adéquates, le projet a été réalisé avec :

- ✓ Ordinateur : Dell Pro 14 (PC14250) ;
- ✓ Processeur : Intel Core TM Ultra 5 235U vProTM (12 TOPS NPU, 12 cœurs, 4,9 GHz) ;
- ✓ OS : Ubuntu 24.04 LTS
- ✓ Système : 64-bit
- ✓ Mémoire vive : 32 Go de RAM.
- ✓ Stockage : 1 TB SSD..

4.1.2 Environnement logiciel et plateforme cloud

Plusieurs outils/logiciels et plateformes ont été exploités pour couvrir les différentes étapes du projet, depuis le développement jusqu'au déploiement.

- ✓ **PostgreSQL** : c'est un système de gestion de base de données relationnelle open source reconnu pour sa robustesse, sa stabilité et ses performances. Il est utilisé dans ce projet comme base de données transactionnelle source pour stocker les don-

nées opérationnelles du système de vente de livres (clients, livres, ventes, factures, catégories). Comme avantages de PostgreSQL est :

- Gratuit et open source ;
- Très fiable pour les applications transactionnelles ;
- Supporte SQL avancé et les relations complexes ;
- Bonne gestion de volumes importants de données ;
- Large communauté et documentation abondante.

PostgreSQL constitue une excellente base source pour simuler un environnement réel de gestion commerciale.

- ✓ **Snowflake** : c'est une plateforme cloud de Data Warehouse moderne conçue pour le stockage, le traitement et l'analyse de grandes quantités de données. Elle repose sur une architecture scalable séparant le stockage et la puissance de calcul. Elle offre plusieurs avantages comme :

- Solution 100% cloud moderne ;
- Très haute performance analytique ;
- Scalabilité automatique ;
- Séparation stockage / calcul ;
- Facilité d'administration ;
- Compatible avec de nombreux outils Big Data.

Snowflake est idéal pour construire un Data Warehouse performant capable d'héberger les couches RAW, STAGGING, WAREHOUSE et MARTS du projet.

- ✓ **DBT : (Data Build Tool)** est un outil moderne de transformation de données permettant de réaliser les traitements directement dans le Data Warehouse à l'aide de requêtes SQL structurées et versionnées. Il présente quelques avantages que sont :

- Basé sur SQL, simple à utiliser ;
- Facilite la modularité des transformations ;
- Documentation automatique ;
- Gestion des dépendances entre modèles ;
- Tests de qualité des données intégrés ;
- Très utilisé en Data Engineering moderne.

DBT permet de transformer efficacement les données RAW vers STAGGING, WAREHOUSE et MARTS avec une logique propre et maintenable.

- ✓ **Apache Airflow** : c'est une plateforme d'orchestration de workflows permettant de planifier, exécuter et superviser automatiquement des pipelines de données. Ces avantages sont entre autres :

- Automatisation complète des pipelines ;
- Planification des traitements ;
- Interface web de supervision ;
- Gestion des dépendances entre tâches ;
- Suivi des erreurs et relances automatiques ;
- Très populaire dans les projets Big Data.

Airflow permet d'automatiser l'enchaînement des tâches du projet : ingestion, exécution DBT et mise à jour du Data Warehouse.

- ✓ **Streamlit** : c'est un framework Python open source permettant de développer rapidement des applications web interactives orientées Data Science et visualisation. Comme avantages, il présente :

- Développement rapide en Python ;
- Interface moderne et interactive ;
- Intégration simple avec Pandas et Plotly ;
- Déploiement facile ;
- Idéal pour les dashboards analytiques.

Streamlit permet de créer rapidement un tableau de bord interactif pour visualiser les KPI du projet BookFlow.

L'association de ces cinq outils permet de construire une chaîne complète de traitement de données moderne, robuste et adaptée aux exigences d'un projet Big Data professionnel.

4.2 Processus de la réalisation

Après la présentation de la première étape dans le chapitre 2, cette section s'attache à la mise en œuvre des étapes suivantes. Elle aborde principalement :

- ✓ la création de la base locale ;
- ✓ l'ingestion vers Snowflake ;
- ✓ la transformation RAW vers STAGGING ;
- ✓ la transformation STAGGING vers WAREHOUSE ;
- ✓ la Création du MART OBT Sales ;
- ✓ l'orchestration avec Airflow ;
- ✓ la visualisation avec Streamlit.

Ces différents points constituent le cœur du processus de réalisation de ce projet.

4.2.1 Création des répertoires et l'architecture du projet

4.2.1.1 Création des répertoires

L'architecture du projet **BOOKSHOP** repose sur une organisation modulaire et structurée des différents composants techniques, facilitant à la fois le développement, la maintenance et l'évolution du système. L'arborescence du projet (tree structure) a été conçue de manière à séparer clairement les différentes couches fonctionnelles du pipeline de données, depuis la source transactionnelle jusqu'à la visualisation finale.

À la racine du projet se trouvent les principaux répertoires stratégiques. Le dossier **scripts/** regroupe les scripts Python dédiés aux opérations d'initialisation et d'ingestion. On y retrouve notamment le script de création de l'environnement Snowflake ainsi que le script d'extraction des données PostgreSQL vers Snowflake. Ce dossier constitue la couche d'acquisition des données. Le dossier *dbt/bookshop_dbt/* héberge le projet dbt chargé des transformations analytiques. Il contient les modèles SQL organisés selon une architecture en couches moderne : sources, staging, warehouse et marts. La couche sources référence les tables brutes importées dans Snowflake. La couche staging permet le nettoyage, le renommage et la normalisation des données. La couche warehouse contient les tables dimensionnelles et factuelles servant à la modélisation décisionnelle. Enfin, la couche marts fournit des tables métier optimisées pour les tableaux de bord et l'analyse finale.

Le dossier **airflow/** contient l'orchestration du pipeline à travers des DAGs Apache Airflow. Cette partie permet d'automatiser l'exécution séquentielle des traitements : ingestion, transformations dbt staging, warehouse et marts. Grâce à cette couche, le projet passe d'un simple ensemble de scripts à une véritable plateforme industrielle de traitement de données. Le dossier **dashboard/** regroupe l'application Streamlit utilisée pour la restitution visuelle des résultats. Cette interface permet de consulter les indicateurs clés (chiffre d'affaires, volumes de ventes, performances clients, tendances annuelles, etc.) à partir des données consolidées stockées dans Snowflake. Les Figures 4.1 et 4.2 présentent respectivement la création des différents répertoires en ligne de commande et l'arborescence du dossier du projet.

```
(base) djjidjooh@djjidjooh-Dell-Pro-14-PC14250:~/Exam_TechBigData$ mkdir Exam_BOOKSHOP
(base) djjidjooh@djjidjooh-Dell-Pro-14-PC14250:~/Exam_TechBigData$ cd Exam_BOOKSHOP
(base) djjidjooh@djjidjooh-Dell-Pro-14-PC14250:~/Exam_TechBigData/Exam_BOOKSHOP$ mkdir sql scripts airflow dashboard dbt
(base) djjidjooh@djjidjooh-Dell-Pro-14-PC14250:~/Exam_TechBigData/Exam_BOOKSHOP$ mkdir airflow/dags
(base) djjidjooh@djjidjooh-Dell-Pro-14-PC14250:~/Exam_TechBigData/Exam_BOOKSHOP$
```

FIGURE 4.1 – Commandes de créations des différents dossiers du projet

```
(base) djidjooh@djidjooh-Dell-Pro-14-PC14250:~/Exam_TechBigData/Exam_BOOKSHOP$ tree
.
├── airflow
│   └── dags
├── dashboard
├── dbt
├── scripts
└── sql

7 directories, 0 files
```

FIGURE 4.2 – Verification des dossiers créés

4.2.1.2 Architecture du projet

L'architecture globale du projet a été gérée dans le fichier **docker-compose.yml**, créé à la racine du projet. Il constitue le point central de déploiement du projet BOOKSHOP. Il permet de définir, configurer et exécuter automatiquement l'ensemble des services nécessaires au fonctionnement de la plateforme dans des conteneurs Docker interconnectés. Dans ce projet, il orchestre notamment la base de données PostgreSQL utilisée comme source transactionnelle, les services Apache Airflow (webserver, scheduler, base metadata) dédiés à l'automatisation du pipeline de données, ainsi que les volumes persistants et les réseaux nécessaires à la communication entre composants. Grâce à ce fichier, l'ensemble de l'environnement technique peut être démarré, arrêté ou redéployé simplement à l'aide de commandes standardisées, garantissant portabilité, reproductibilité et simplicité d'installation.

Dans le cadre de ce projet, ce fichier facilite la mise en place d'une architecture reproductible. Un utilisateur peut cloner le dépôt GitHub, renseigner les variables d'environnement nécessaires, puis lancer toute la plateforme avec une simple commande. Cette approche est particulièrement importante dans un projet Big Data, car elle garantit que le pipeline fonctionne de la même manière sur différentes machines, indépendamment du système hôte.

Ainsi, l'architecture retenue pour BOOKSHOP reflète les standards modernes du Data Engineering, en combinant ingestion, transformation, orchestration, stockage cloud et visualisation décisionnelle dans une solution cohérente, robuste et évolutive.

Les Figures 4.3 et 4.4 montrent les captures d'écran de la création du fichier **docker-compose.yml** et de son exécution. Tout le dossier du projet est accessible via le lien : <https://github.com/Djidjooh/bookflow-dashboard> où le fichier **docker-compose.yml** est disponible.

Cette étape ayant été accomplie avec succès, la suite a été la création de la base de données locale avec PostgreSQL

```
GNU nano 7.2 docker-compose.yml

services:

  postgres:
    image: postgres:16
    container_name: bookflow_postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./sql:/docker-entrypoint-initdb.d
    networks:
      - bookflow_net

  airflow-db:
    image: postgres:16
    container_name: bookflow_airflow_db
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - airflow_db_data:/var/lib/postgresql/data
    networks:
      - bookflow_net

  airflow-init:
    image: apache/airflow:2.9.3
    container_name: bookflow_airflow_init
    depends_on:
      - airflow-db
    environment:
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@airflow-db:5432/airflow
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__CORE__LOAD_EXAMPLES: "false"
      AIRFLOW__WEBSERVER__SECRET_KEY: "bookflow_secret_key_2026"
      _PIP_ADDITIONAL_REQUIREMENTS: "pandas psycopg2-binary snowflake-connector-python dbt-core dbt-snowflake"

      # Variables PostgreSQL accessibles depuis les scripts Airflow
      POSTGRES_HOST: postgres
      POSTGRES_PORT: 5432
      POSTGRES_DB: ${POSTGRES_DB}
```

FIGURE 4.3 – Screenshot de la création du fichier docker-compose.yml

```
(base) djdjoo@djdjoo-Dell-Pro-14-PC14250: /Exam_TechBigData/Exam_BOOKSHOP$ sudo systemctl restart docker
[sudo] password for djdjoo:
Warning: The unit file, source configuration file or drop-ins of docker.service changed on disk. Run 'systemctl daemon-reload' to reload units.
(base) djdjoo@djdjoo-Dell-Pro-14-PC14250: /Exam_TechBigData/Exam_BOOKSHOP$ sudo iptables -N DOCKER-ISOLATION-STAGE-1
(base) djdjoo@djdjoo-Dell-Pro-14-PC14250: /Exam_TechBigData/Exam_BOOKSHOP$ sudo iptables -N DOCKER-ISOLATION-STAGE-2
(base) djdjoo@djdjoo-Dell-Pro-14-PC14250: /Exam_TechBigData/Exam_BOOKSHOP$ sudo iptables -A DOCKER-ISOLATION-STAGE-1 -j DOCKER-ISOLATION-STAGE-2
(base) djdjoo@djdjoo-Dell-Pro-14-PC14250: /Exam_TechBigData/Exam_BOOKSHOP$ sudo iptables -A DOCKER-ISOLATION-STAGE-2 -j RETURN
(base) djdjoo@djdjoo-Dell-Pro-14-PC14250: /Exam_TechBigData/Exam_BOOKSHOP$ docker-compose up -d
[+] Running 10/10
 ✓ Network exam_bookshop_bookflow_net      Created
 ✓ Volume "exam_bookshop_airflow_db_data"  Created
 ✓ Volume "exam_bookshop_postgres_data"    Created
 ✓ Container bookflow_postgres              Started
 ✓ Container bookflow_airflow_db            Started
 ✓ Container bookflow_streamlit             Started
 ✓ Container bookflow_dbt                   Started
 ✓ Container bookflow_airflow_init          Started
 ✓ Container bookflow_airflow_webserver     Started
 ✓ Container bookflow_airflow_scheduler     Started
(base) djdjoo@djdjoo-Dell-Pro-14-PC14250: /Exam_TechBigData/Exam_BOOKSHOP$
```

FIGURE 4.4 – Screenshot montrant la création des conteneurs nécessaires du projet

4.2.2 Création de la base locale PostgreSQL

Dans le cadre du projet **BOOKSHOP**, une base de données locale **PostgreSQL** a été mise en place afin de simuler le système opérationnel de vente de livres. Cette base constitue la source primaire des données exploitées dans le pipeline Big Data. La création de cette base locale permet de reproduire un environnement réel dans lequel les transactions commerciales sont enregistrées quotidiennement. Elle contient les principales tables métier nécessaires au projet, notamment :

- ✓ **customers** : informations sur les clients ;
- ✓ **books** : catalogue des livres disponibles ;
- ✓ **category** : catégories de livres ;
- ✓ **factures** : données relatives aux factures émises ;
- ✓ **ventes** : enregistrements détaillés des ventes réalisées.

```
GNU nano 7.2
CREATE TABLE category (
  id SERIAL PRIMARY KEY,
  code VARCHAR(50),
  intitule VARCHAR(255)
);

CREATE TABLE books (
  id SERIAL PRIMARY KEY,
  code VARCHAR(50),
  intitule VARCHAR(255),
  isbn_10 VARCHAR(20),
  isbn_13 VARCHAR(20),
  category_id INT REFERENCES category(id)
);

CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  code VARCHAR(50),
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  email VARCHAR(150)
);

CREATE TABLE factures (
  id SERIAL PRIMARY KEY,
  code VARCHAR(50),
  customer_id INT REFERENCES customers(id),
  date_edit VARCHAR(8),
  qte_totale INT,
  total_amount NUMERIC(12,2),
  total_paid NUMERIC(12,2)
);

CREATE TABLE ventes (
  id SERIAL PRIMARY KEY,
  facture_id INT REFERENCES factures(id),
  book_id INT REFERENCES books(id),
  date_edit VARCHAR(8),
  pu NUMERIC(12,2),
  qte INT
);
```

FIGURE 4.5 – Création des tables

```
bookshop=# SELECT * FROM category LIMIT 5;
 id | code |   intitule
-----+-----+-----
  1 | CAT001 | Informatique
  2 | CAT002 | Big Data
  3 | CAT003 | Science des données
  4 | CAT004 | Programmation
  5 | CAT005 | Machine Learning
(5 rows)

bookshop=# SELECT * FROM books LIMIT 5;
 id | code |   intitule   | isbn_10 | isbn_13 | category_id
-----+-----+-----+-----+-----+-----
  1 | BK001 | Introduction au Big Data | 1234567001 | 9781234567001 | 1
  2 | BK002 | Python pour Data Science | 1234567002 | 9781234567002 | 2
  3 | BK003 | Bases de données modernes | 1234567003 | 9781234567003 | 3
  4 | BK004 | SQL Avancé | 1234567004 | 9781234567004 | 4
  5 | BK005 | Machine Learning Fondamental | 1234567005 | 9781234567005 | 5
(5 rows)

bookshop=# SELECT * FROM customers LIMIT 5;
 id | code | first_name | last_name | email
-----+-----+-----+-----+-----
  1 | CL001 | Maurice | AHOUANSOU | maurice@email.com
  2 | CL002 | Jean | KOFFI | jean@email.com
  3 | CL003 | Amina | SANI | amina@email.com
  4 | CL004 | Paul | MENSAH | paul@email.com
  5 | CL005 | Sonia | DOSSOU | sonia@email.com
(5 rows)

bookshop=# SELECT * FROM factures LIMIT 5;
 id | code | customer_id | date_edit | qte_totale | total_amount | total_paid
-----+-----+-----+-----+-----+-----+-----
  1 | FAC001 | 1 | 20250301 | 2 | 30000.00 | 30000.00
  2 | FAC002 | 2 | 20250302 | 1 | 15000.00 | 10000.00
  3 | FAC003 | 3 | 20250303 | 3 | 45000.00 | 45000.00
  4 | FAC004 | 4 | 20250304 | 2 | 30000.00 | 30000.00
  5 | FAC005 | 5 | 20250305 | 4 | 60000.00 | 60000.00
(5 rows)

bookshop=# SELECT * FROM ventes LIMIT 5;
 id | facture_id | book_id | date_edit | pu | qte
-----+-----+-----+-----+-----+-----
  1 | 1 | 1 | 20250301 | 15000.00 | 1
  2 | 2 | 2 | 20250302 | 15000.00 | 1
  3 | 3 | 3 | 20250303 | 15000.00 | 3
  4 | 4 | 4 | 20250304 | 15000.00 | 2
  5 | 5 | 5 | 20250305 | 15000.00 | 4
(5 rows)
```

FIGURE 4.6 – Vérification des données

Chaque table a été conçue avec des clés primaires et étrangères afin de respecter l'intégrité relationnelle des données. Des données de test réalistes ont ensuite été insérées pour alimenter les traitements analytiques futurs. Le choix de PostgreSQL comme base locale se justifie par sa robustesse, sa conformité aux standards SQL, sa gratuité ainsi que sa capacité à gérer efficacement les bases relationnelles transactionnelles. Cette étape représente le point de départ du projet, car elle fournit les données brutes qui seront ensuite extraites, transférées vers Snowflake, transformées avec DBT puis valorisées dans

les tableaux de bord décisionnels. La figure 4.5 présente le script utilisé pour la création des tables dans PostgreSQL.

Après la création des différentes tables, il a été procédé à l'insertion des données dans chaque table. A cet effet, 100 lignes d'informations ont été générées et insérées dans les tables. La Figure 4.6 montre les résultats du test de vérification de l'existence des données dans les différentes tables.

4.2.3 Ingestion vers Snowflake

L'étape d'ingestion constitue un maillon central de l'architecture décisionnelle mise en place dans le cadre du projet BOOKSHOP. Elle a consisté à transférer les données opérationnelles stockées dans la base PostgreSQL vers l'environnement analytique Snowflake, qui joue le rôle d'entrepôt de données cloud. Cette phase a permis de centraliser les informations sources dans un espace scalable, performant et adapté aux traitements analytiques.

Cette étape permet d'assurer l'alimentation du schéma RAW de Snowflake avec les données extraites de PostgreSQL, sans altération majeure de leur structure initiale.

Le schéma RAW constitue ainsi une zone d'atterrissage (landing zone) où les données sont stockées dans leur état brut avant les traitements de transformation réalisés ultérieurement avec dbt. Les tables concernées par cette ingestion sont les suivantes :

- ✓ **CATEGORY** ;
- ✓ **BOOKS** ;
- ✓ **customers** ;
- ✓ **FACTURES** ;
- ✓ **VENTES**.

Ces tables représentent respectivement les catégories d'ouvrages, les livres, les clients, les factures et les ventes réalisées.

Cette tâche est exécutée avant les transformations dbt, garantissant ainsi que les données les plus récentes soient disponibles dans Snowflake avant tout traitement.

L'ingestion vers Snowflake a permis de mettre en place une passerelle fiable entre la base transactionnelle PostgreSQL et l'entrepôt analytique cloud. Elle représente la première étape du pipeline ELT développé dans ce projet, assurant disponibilité, traçabilité et centralisation des données nécessaires aux analyses métier et au tableau de bord décisionnel. La Figure 4.7 présente le processus d'ingestion des données dans Snowflake. Les Figures 4.8 et 4.9 montrent d'une part quelques captures d'écran partielles des scripts utilisés pour la création des tables et d'ingestion des données de PostgreSQL dans Snowflake et d'autre part la vérification de la disponibilité des données injectées sur la plateforme de Snowflake.

PROCESSUS D'INGESTION VERS SNOWFLAKE

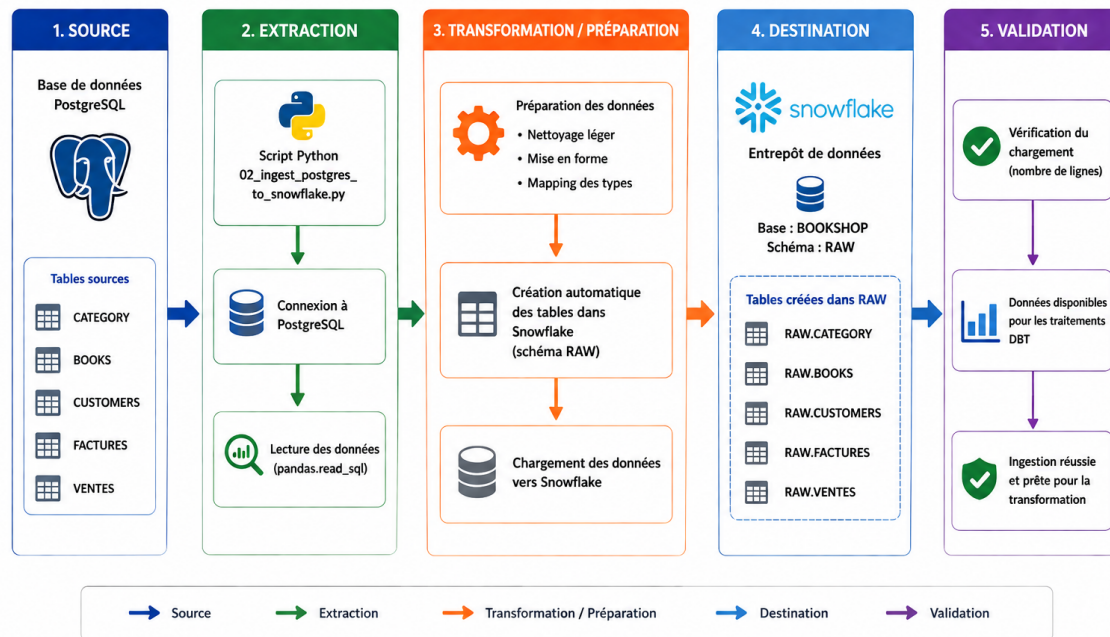


FIGURE 4.7 – Flowchart du processus d'ingestion des données dans Snowflake

```

scripts > 01_setup_snowflake.py > ...
1 import os
2 import snowflake.connector
3
4 conn = snowflake.connector.connect(
5     user=os.getenv("SNOWFLAKE_USER"),
6     password=os.getenv("SNOWFLAKE_PASSWORD"),
7     account=os.getenv("SNOWFLAKE_ACCOUNT"),
8     role=os.getenv("SNOWFLAKE_ROLE", "ACCOUNTADMIN"),
9 )
10
11 cur = conn.cursor()
12
13 queries = [
14     """
15     CREATE WAREHOUSE IF NOT EXISTS COMPUTE_WH
16     WITH WAREHOUSE_SIZE = 'XSMALL'
17     AUTO_SUSPEND = 60
18     AUTO_RESUME = TRUE
19     """
20     ,
21     "CREATE DATABASE IF NOT EXISTS BOOKSHOP",
22     "CREATE SCHEMA IF NOT EXISTS BOOKSHOP.RAW",
23     "CREATE SCHEMA IF NOT EXISTS BOOKSHOP.STAGGING",
24     "CREATE SCHEMA IF NOT EXISTS BOOKSHOP.WAREHOUSE",
25     "CREATE SCHEMA IF NOT EXISTS BOOKSHOP.MARTS",
26 ]
27
28 for query in queries:
29     print(f"Exécution : {query}")
30     cur.execute(query)
31
32 cur.close()
33 conn.close()
34
35 print("✅ Base BOOKSHOP et schémas créés avec succès dans Snowflake.")
36
37
scripts > 02_ingest_postgres_to_snowflake.py > ...
1 import warnings
2 warnings.filterwarnings("ignore")
3 import os
4 import pandas as pd
5 import psycopg2
6 import snowflake.connector
7 from snowflake.connector.pandas_tools import write_pandas
8
9 POSTGRES_CONFIG = {
10     "host": "postgres",
11     "port": 5432,
12     "database": os.getenv("POSTGRES_DB", "bookshop"),
13     "user": os.getenv("POSTGRES_USER", "bookuser"),
14     "password": os.getenv("POSTGRES_PASSWORD", "bookpass"),
15 }
16
17 SNOWFLAKE_CONFIG = {
18     "account": os.getenv("SNOWFLAKE_ACCOUNT"),
19     "user": os.getenv("SNOWFLAKE_USER"),
20     "password": os.getenv("SNOWFLAKE_PASSWORD"),
21     "role": os.getenv("SNOWFLAKE_ROLE", "ACCOUNTADMIN"),
22     "warehouse": os.getenv("SNOWFLAKE_WAREHOUSE", "COMPUTE_WH"),
23     "database": os.getenv("SNOWFLAKE_DATABASE", "BOOKSHOP"),
24     "schema": "RAW",
25 }
26
27 TABLES = ["category", "books", "customers", "factures", "ventes"]
28
29 def postgres_to_dataframe(table_name, pg_conn):
30     query = f"SELECT * FROM {table_name};"
31     print(f"Lecture PostgreSQL : {table_name}")
32     return pd.read_sql(query, pg_conn)
33
34

```

FIGURE 4.8 – Script de creation des tables et ingestion dans Snowflake

Le processus d'ingestion vers Snowflake a permis d'établir un lien robuste entre la base de données transactionnelle PostgreSQL et l'entrepôt de données analytique hébergé dans le cloud. Il constitue la phase initiale du pipeline ELT mis en œuvre dans ce projet, garantissant la disponibilité, la centralisation ainsi que la traçabilité des données indispensables aux analyses décisionnelles et à l'alimentation du tableau de bord métier.

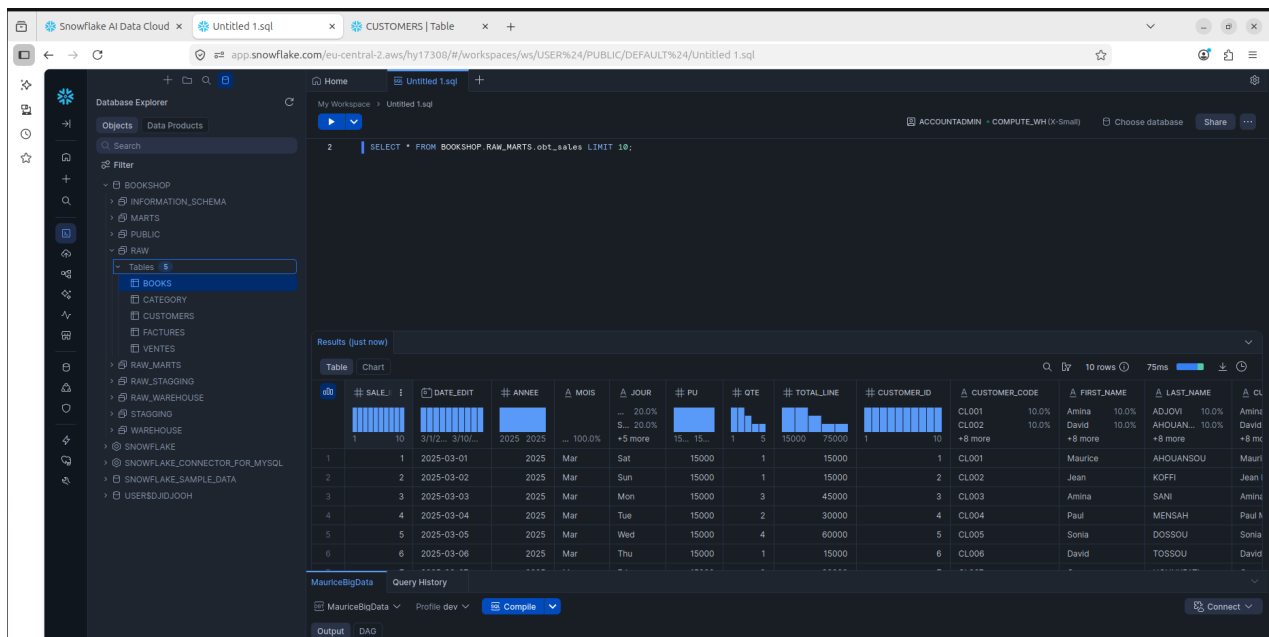


FIGURE 4.9 – Verification de la presence des donnee dans Snowflake

4.2.4 Transformation RAW vers STAGGING

L'étape de transformation RAW vers STAGGING représente la première phase de traitement des données au sein du pipeline analytique du projet BOOKSHOP. Après l'ingestion des données brutes depuis PostgreSQL vers le schéma RAW de Snowflake, les informations importées conservent généralement leur structure opérationnelle d'origine. Elles peuvent contenir des noms de colonnes peu explicites, des formats hétérogènes, des types de données non optimisés ou des valeurs nécessitant une standardisation. La couche STAGGING a donc été mise en place afin de préparer ces données pour les traitements décisionnels ultérieurs. Cette transformation a été réalisée à l'aide de l'outil dbt (Data Build Tool), permettant de modéliser les traitements sous forme de requêtes SQL versionnées et facilement maintenables. Plusieurs modèles ont été développés pour chaque table source, notamment :

- ✓ **stg_books** ;
- ✓ **stg_category** ;
- ✓ **stg_customers** ;
- ✓ **stg_factures** ;
- ✓ **stg_ventes**.

Chaque modèle extrait les données depuis le schéma RAW puis applique un ensemble de règles de préparation.

Les principales opérations effectuées dans cette couche sont le renommage des colonnes selon des conventions plus explicites, la conversion des types de données, la suppression

d'éventuelles incohérences, ainsi que l'harmonisation des formats de dates et de champs numériques. Par exemple, certaines colonnes techniques issues du système source ont été renommées afin de faciliter la compréhension métier et l'exploitation analytique future.

La couche STAGGING joue également un rôle important en matière de traçabilité et de séparation des responsabilités. Elle permet de conserver intactes les données originales dans RAW tout en offrant une version propre, cohérente et standardisée des tables utilisées par les modèles de niveau supérieur. Cette approche respecte les bonnes pratiques modernes de l'architecture ELT.

L'exécution de cette étape a été automatisée via la commande : **dbt run --select staging**. La Figure 4.10 montre la réussite de cette étape.

```
root@0ef13f5fb3c6:/app/bookshop_dbt# dbt run --select staging
15:00:01 Running with dbt=1.11.8
15:00:01 Registered adapter: snowflake=1.11.4
15:00:01 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.bookshop_dbt.example
15:00:01 Found 5 models, 5 sources, 523 macros
15:00:01
15:00:01 Concurrency: 4 threads (target='dev')
15:00:01
15:00:05 1 of 5 START sql table model RAW_STAGGING.stg_books ..... [RUN]
15:00:05 2 of 5 START sql table model RAW_STAGGING.stg_category ..... [RUN]
15:00:05 3 of 5 START sql table model RAW_STAGGING.stg_customers ..... [RUN]
15:00:05 4 of 5 START sql table model RAW_STAGGING.stg_factures ..... [RUN]
15:00:07 1 of 5 OK created sql table model RAW_STAGGING.stg_books ..... [SUCCESS 100 in 1.96s]
15:00:07 2 of 5 OK created sql table model RAW_STAGGING.stg_category ..... [SUCCESS 100 in 1.96s]
15:00:07 5 of 5 START sql table model RAW_STAGGING.stg_ventes ..... [RUN]
15:00:08 4 of 5 OK created sql table model RAW_STAGGING.stg_factures ..... [SUCCESS 100 in 2.12s]
15:00:08 3 of 5 OK created sql table model RAW_STAGGING.stg_customers ..... [SUCCESS 100 in 2.13s]
15:00:08 5 of 5 OK created sql table model RAW_STAGGING.stg_ventes ..... [SUCCESS 100 in 0.98s]
15:00:10
15:00:10 Finished running 5 table models in 0 hours 0 minutes and 8.46 seconds (8.46s).
15:00:10
15:00:10 Completed successfully
15:00:10
15:00:10 Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=5
```

FIGURE 4.10 – Screenshot de la transformation RAW vers STAGGING

En bref, la transformation RAW vers STAGGING constitue une étape fondamentale du projet. Elle améliore la qualité des données, simplifie les traitements suivants et prépare efficacement les jeux de données destinés à la modélisation du Data Warehouse et à l'analyse décisionnelle.

4.2.5 Transformation STAGGING vers WAREHOUSE

L'étape de transformation STAGGING vers WAREHOUSE constitue la deuxième phase majeure du pipeline analytique développé dans le cadre du projet BookFlow Analytics Platform. Après la préparation et la standardisation des données dans la couche STAGGING, cette étape vise à structurer les informations selon une logique décisionnelle

adaptée aux analyses métier, à la production d'indicateurs de performance et à l'alimentation du tableau de bord.

La couche WAREHOUSE correspond au cœur de l'entrepôt de données. Elle regroupe des tables organisées selon les principes de modélisation dimensionnelle, permettant d'optimiser les requêtes analytiques et de faciliter l'exploration des données. À ce niveau, les données préparées dans STAGGING sont enrichies, reliées entre elles et réparties entre tables de dimensions et tables de faits.

Dans le projet BookFlow, plusieurs modèles ont été mis en place, notamment :

- ✓ **dim_books** ;
- ✓ **dim_category** ;
- ✓ **dim_customers** ;
- ✓ **fact_sales** ;
- ✓ **fact_factures** ;
- ✓ **fact_ventes**.

Les tables de dimensions contiennent les informations descriptives relativement stables, telles que les livres, les clients ou les catégories. Elles permettent de contextualiser les analyses. À l'inverse, les tables de faits centralisent les événements mesurables comme les ventes, les montants facturés ou les quantités écoulées.

Les principales opérations réalisées lors de cette transformation sont :

- ✓ création des clés analytiques ;
- ✓ jointures entre tables préparées ;
- ✓ consolidation des transactions ;
- ✓ calcul de mesures (quantité vendue, chiffre d'affaires, total ligne) ;
- ✓ structuration temporelle des données ;
- ✓ optimisation du schéma pour les analyses rapides.

Cette étape a été implémentée avec dbt, garantissant la traçabilité des transformations et la reproductibilité des traitements. L'exécution est réalisée à l'aide de la commande : **run --select warehouse --profiles-dir /tmp/.dbt**. La Figure 4.11 montre la réussite de cette étape.

```

airflow@1e2c9abf1e6d:/opt/airflow/dbt/bookshop_dbt$ dbt run --select warehouse --profiles-dir /tmp/.dbt
03:33:36 Running with dbt=1.11.8
03:33:38 Registered adapter: snowflake=1.11.4
03:33:38 Found 15 models, 5 sources, 524 macros
03:33:38
03:33:38 Concurrency: 4 threads (target='dev')
03:33:38
03:33:41 1 of 9 START sql table model WAREHOUSE.dim_books ..... [RUN]
03:33:41 2 of 9 START sql table model WAREHOUSE.dim_category ..... [RUN]
03:33:41 3 of 9 START sql table model WAREHOUSE.dim_customers ..... [RUN]
03:33:41 4 of 9 START sql table model WAREHOUSE.fact_factures ..... [RUN]
03:33:43 3 of 9 OK created sql table model WAREHOUSE.dim_customers ..... [SUCCESS 100 in 1.41s]
03:33:43 4 of 9 OK created sql table model WAREHOUSE.fact_factures ..... [SUCCESS 100 in 1.42s]
03:33:43 1 of 9 OK created sql table model WAREHOUSE.dim_books ..... [SUCCESS 100 in 1.41s]
03:33:43 2 of 9 OK created sql table model WAREHOUSE.dim_category ..... [SUCCESS 100 in 1.40s]
03:33:43 5 of 9 START sql table model WAREHOUSE.fact_sales ..... [RUN]
03:33:43 6 of 9 START sql table model WAREHOUSE.fact_ventes ..... [RUN]
03:33:44 6 of 9 OK created sql table model WAREHOUSE.fact_ventes ..... [SUCCESS 100 in 0.70s]
03:33:44 5 of 9 OK created sql table model WAREHOUSE.fact_sales ..... [SUCCESS 100 in 0.71s]
03:33:44 7 of 9 START sql table model WAREHOUSE.fact_books_annees ..... [RUN]
03:33:44 8 of 9 START sql table model WAREHOUSE.fact_books_jour ..... [RUN]
03:33:44 9 of 9 START sql table model WAREHOUSE.fact_books_mois ..... [RUN]
03:33:44 9 of 9 OK created sql table model WAREHOUSE.fact_books_mois ..... [SUCCESS 100 in 0.82s]
03:33:45 8 of 9 OK created sql table model WAREHOUSE.fact_books_jour ..... [SUCCESS 100 in 0.83s]
03:33:45 7 of 9 OK created sql table model WAREHOUSE.fact_books_annees ..... [SUCCESS 100 in 0.84s]
03:33:46
03:33:46 Finished running 9 table models in 0 hours 0 minutes and 7.57 seconds (7.57s).
03:33:46 Completed successfully
03:33:46
03:33:46 Done PASS=9 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=9

```

FIGURE 4.11 – Screenshot de la transformation STAGGING vers WAREHOUSE

Dans l’orchestration Airflow, cette tâche intervient automatiquement après la couche STAGGING, assurant ainsi la cohérence du pipeline de données.

L’intérêt principal de la couche WAREHOUSE réside dans sa capacité à fournir une base robuste et performante pour les analyses décisionnelles. Elle réduit la complexité des requêtes en proposant des structures prêtes à l’emploi, tout en améliorant les temps de réponse sur de grands volumes de données.

En résumé, la transformation STAGGING vers WAREHOUSE permet de passer d’une donnée simplement nettoyée à une donnée véritablement modélisée pour la Business Intelligence. Elle représente une étape stratégique dans la valorisation des données du projet BOOKSHOP.

4.2.6 Création du MART OBT Sales

L’étape de création du MART OBT Sales représente la phase finale de structuration des données dans le pipeline analytique du projet BOOKSHOP. Après les transformations successives réalisées dans les couches RAW, STAGGING et WAREHOUSE, cette étape consiste à produire une table métier consolidée, directement exploitable pour l’analyse décisionnelle et la visualisation dans le tableau de bord.

Le terme OBT (One Big Table) désigne une table unique regroupant, dans un même ensemble de données, les principales informations nécessaires à l’analyse des ventes. Contrai-

rement à une modélisation en étoile nécessitant plusieurs jointures entre tables de faits et dimensions, l'approche OBT vise à simplifier l'accès aux données en centralisant les indicateurs clés dans une structure unique, optimisée pour les outils de reporting.

Dans le cadre de ce projet, la table OBT_SALES a été construite à partir des tables de dimensions et de faits de la couche WAREHOUSE, notamment :

- ✓ **dim_books** ;
- ✓ **dim_category** ;
- ✓ **dim_customers** ;
- ✓ **fact_sales** ;
- ✓ **fact_factures** ;
- ✓ **fact_ventes**.

Les jointures entre ces différentes tables ont permis de reconstituer une vue complète des transactions commerciales, en associant les informations produits, clients, catégories et données financières.

La table MART contient ainsi plusieurs champs analytiques tels que :

- ✓ identifiant de vente ;
- ✓ nom du client ;
- ✓ titre du livre ;
- ✓ catégorie du produit ;
- ✓ quantité vendue ;
- ✓ prix unitaire ;
- ✓ montant total ;
- ✓ année, mois et jour de la transaction.

Cette structuration facilite considérablement la création de tableaux de bord interactifs, car toutes les informations utiles sont disponibles dans une seule requête SQL. La génération de cette table a été réalisée à l'aide de dbt, via un modèle SQL dédié : **models/-marts/obt_sales.sql**. Son exécution s'effectue avec la commande : **dbt run --select marts --profiles-dir /tmp/.dbt**. La Figure 4.12 montre la réussite de cette étape.

```

airflow@1e2c9abf1e6d:/opt/airflow/dbt/bookshop_dbt$ dbt run --select marts --profiles-dir /tmp/.dbt
03:56:56 Running with dbt=1.11.8
03:56:56 Registered adapter: snowflake=1.11.4
03:56:56 Found 15 models, 5 sources, 524 macros
03:56:56
03:56:56 Concurrency: 4 threads (target='dev')
03:56:56
03:57:00 1 of 1 START sql table model MARTS.obt_sales ..... [RUN]
03:57:02 1 of 1 OK created sql table model MARTS.obt_sales ..... [SUCCESS 1000 in 1.77s]
03:57:03
03:57:03 Finished running 1 table model in 0 hours 0 minutes and 6.39 seconds (6.39s).
03:57:03
03:57:03 Completed successfully
03:57:03
03:57:03 Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=1

```

FIGURE 4.12 – Screenshot de Création du MART OBT Sales

Dans Apache Airflow, cette étape intervient automatiquement à la fin du pipeline, après la mise à jour des tables du Data Warehouse.

L'intérêt principal du MART OBT Sales réside dans ses performances analytiques et sa simplicité d'utilisation. Il permet de produire rapidement des indicateurs stratégiques tels que le chiffre d'affaires, les meilleures ventes, la performance par année, les clients les plus rentables ou encore les tendances commerciales.

Bref, la création du MART OBT Sales constitue l'aboutissement du processus ELT du projet BOOKSHOP. Elle transforme les données techniques en une ressource directement exploitable par les décideurs, renforçant ainsi la valeur métier de la plateforme analytique développée.

4.2.7 Orchestration avec Airflow

L'orchestration des traitements a été assurée à l'aide de Apache Airflow, outil de référence largement utilisé dans les environnements Data Engineering pour planifier, automatiser et superviser les pipelines de données. Dans le cadre du projet BOOKSHOP, Airflow a permis de coordonner l'ensemble des étapes du processus ELT, depuis l'ingestion des données sources jusqu'à la production finale des tables analytiques.

L'un des principaux avantages d'Airflow réside dans sa capacité à représenter les traitements sous forme de DAG (Directed Acyclic Graph), c'est-à-dire un graphe orienté décrivant l'enchaînement logique des tâches. Chaque tâche dépend de la réussite de la précédente, ce qui garantit la cohérence du pipeline et évite l'exécution de transformations sur des données incomplètes ou non mises à jour.

Dans ce projet, un DAG principal nommé : **bookflow_pipeline** a été développé afin d'automatiser les opérations suivantes :

- ✓ Ingestion PostgreSQL vers Snowflake RAW ;
- ✓ Transformation RAW vers STAGGING ;
- ✓ Transformation STAGGING vers WAREHOUSE ;
- ✓ Création du MART OBT Sales.

La Figure 4.12 montre les processus d'ingestion vers Snowflake et de l'orchestration avec Airflow.

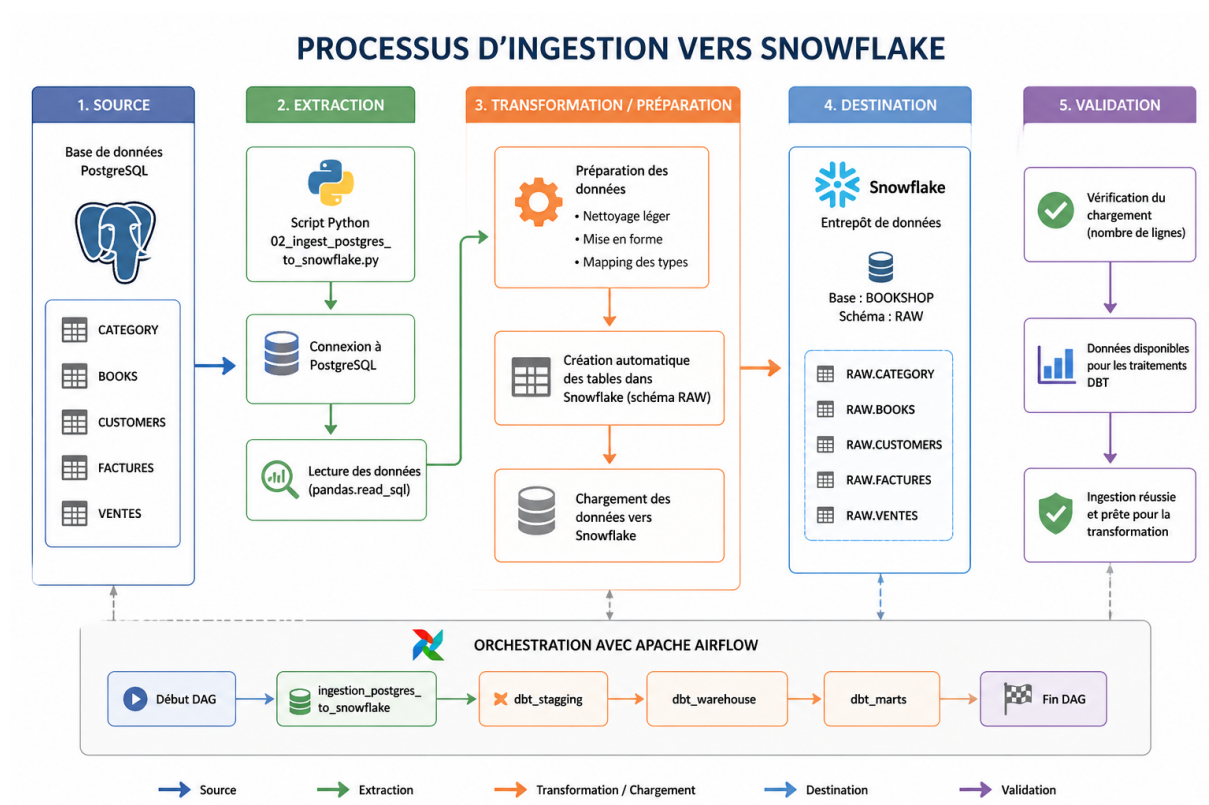


FIGURE 4.13 – Flowchart de l'ingestion vers Snowflake et de l'orchestration avec Airflow

Chaque étape est exécutée séquentiellement selon les dépendances définies dans le DAG. Ainsi, la couche STAGGING n'est lancée qu'après la réussite de l'ingestion, puis la couche WAREHOUSE est déclenchée automatiquement, suivie de la création du MART final comme le montre la Figure 4.14.



FIGURE 4.14 – DAG d'orchestration avec Airflow

La Figure 4.15 présente une capture d'écran de le l'infarce Airflow avec la réussite de l'orchestration.

L'intégration d'Airflow a permis de transformer un ensemble de scripts indépendants en une chaîne de traitement industrielle, fiable et reproductible. Au lieu d'exécuter manuellement plusieurs commandes Python et dbt, un simple déclenchement du DAG suffit pour lancer tout le pipeline.

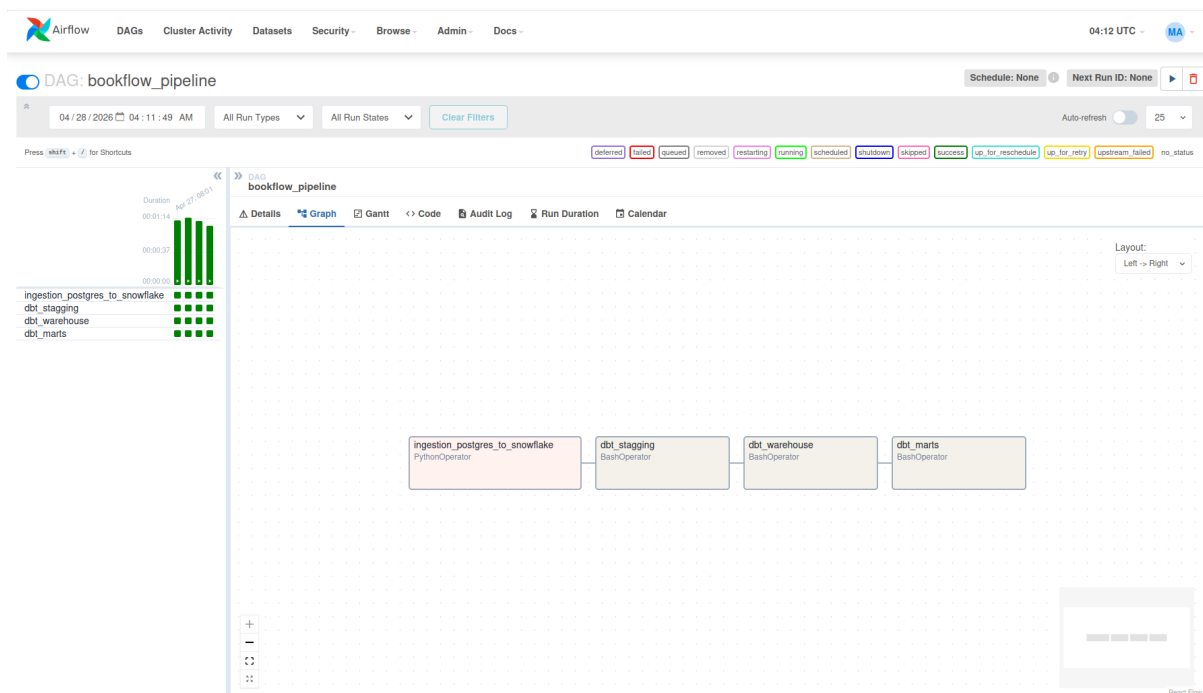


FIGURE 4.15 – Interface Airflow montrant la réussite de l’orchestration

Sur le plan technique, Airflow a été déployé dans des conteneurs Docker, ce qui facilite sa portabilité, sa maintenance et son intégration avec les autres composants du projet.

Enfin, il est à retenir que l’orchestration avec Apache Airflow constitue un élément stratégique du projet BOOKSHOP. Elle garantit l’automatisation, la robustesse et la traçabilité du pipeline de données.

4.2.8 visualisation avec Streamlit

La visualisation des données a été réalisée avec Streamlit, un framework Python permettant de développer rapidement des applications web interactives orientées analyse de données. Dans le cadre du projet BookFlow Analytics Platform, Streamlit a été utilisé pour concevoir un tableau de bord dynamique connecté à la table finale MARTS.OBT_SALES hébergée dans Snowflake.

Cette étape permet de transformer les données consolidées en indicateurs visuels facilement interprétables par les utilisateurs. Le dashboard présente notamment le chiffre d’affaires global, le nombre de ventes, les quantités vendues, les meilleurs clients, les livres les plus vendus ainsi que l’évolution des ventes par année et par période. Les Figures 4.16, 4.17, 4.18 et 4.19 ,montrent quelques captures d’écran du dashboard créé pour ce projet.

L’application Streamlit interroge directement Snowflake afin de récupérer les données issues du MART analytique. Les résultats sont ensuite affichés sous forme de KPI, tableaux

interactifs et graphiques variés tels que des histogrammes, courbes, diagrammes circulaires et graphiques en barres. Cette diversité de visualisations facilite l'analyse des tendances commerciales et l'identification rapide des performances clés.

Le tableau de bord a également été déployé en ligne, ce qui permet d'y accéder depuis un autre ordinateur à travers un simple lien web. Cette fonctionnalité renforce la valeur pratique du projet, car les résultats peuvent être consultés sans installer l'environnement technique complet.

Ainsi, Streamlit constitue la couche finale de valorisation du pipeline BOOKSHOP. Il rend les données exploitables pour la prise de décision et démontre concrètement l'intérêt de l'architecture mise en place, depuis l'ingestion jusqu'à la restitution visuelle des indicateurs métier.

<https://bookflow-dashboard-axmsqedl6wy64edbdwbzxt.streamlit.app/>

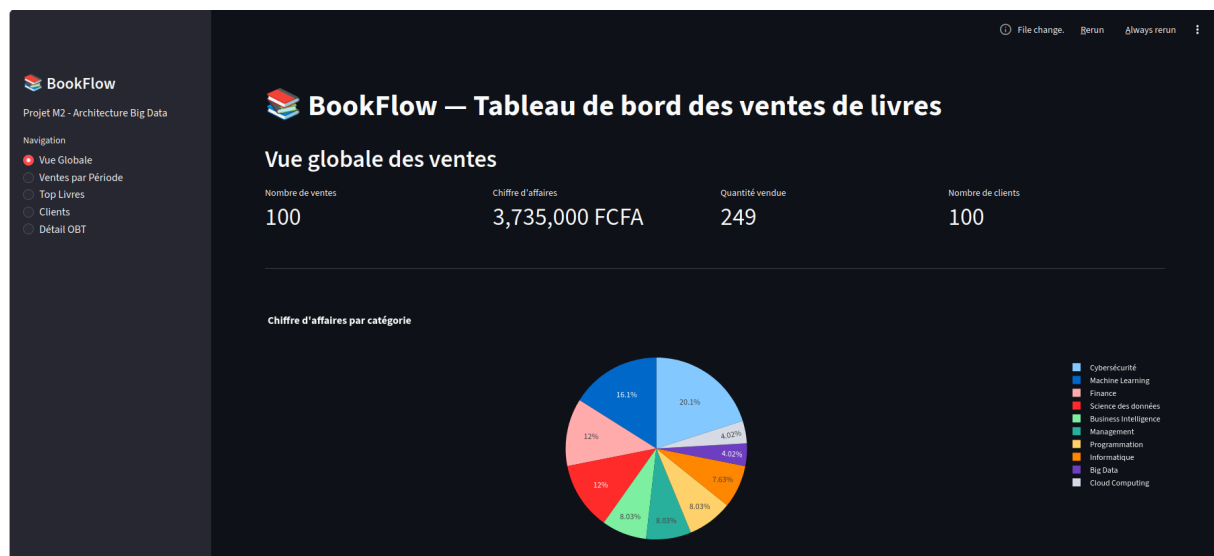


FIGURE 4.16 – Dashboard montrant les ventes globales

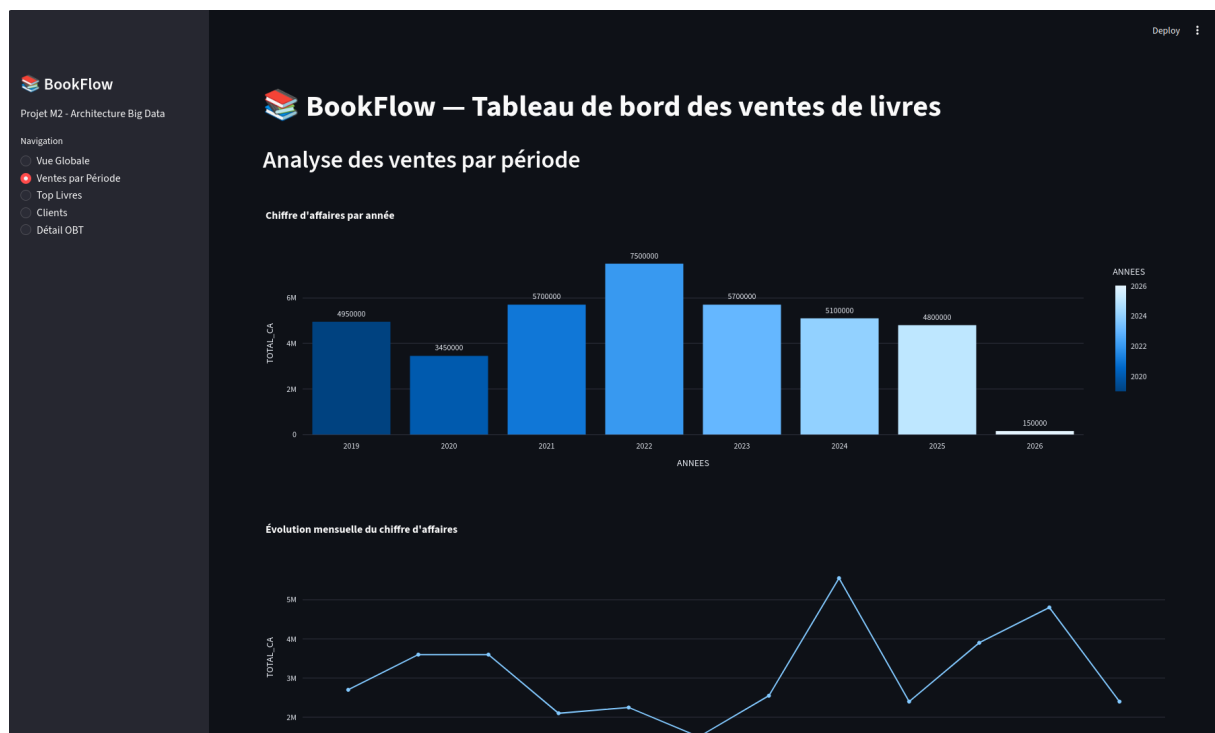


FIGURE 4.17 – Dashboard montrant les ventes de livres par année

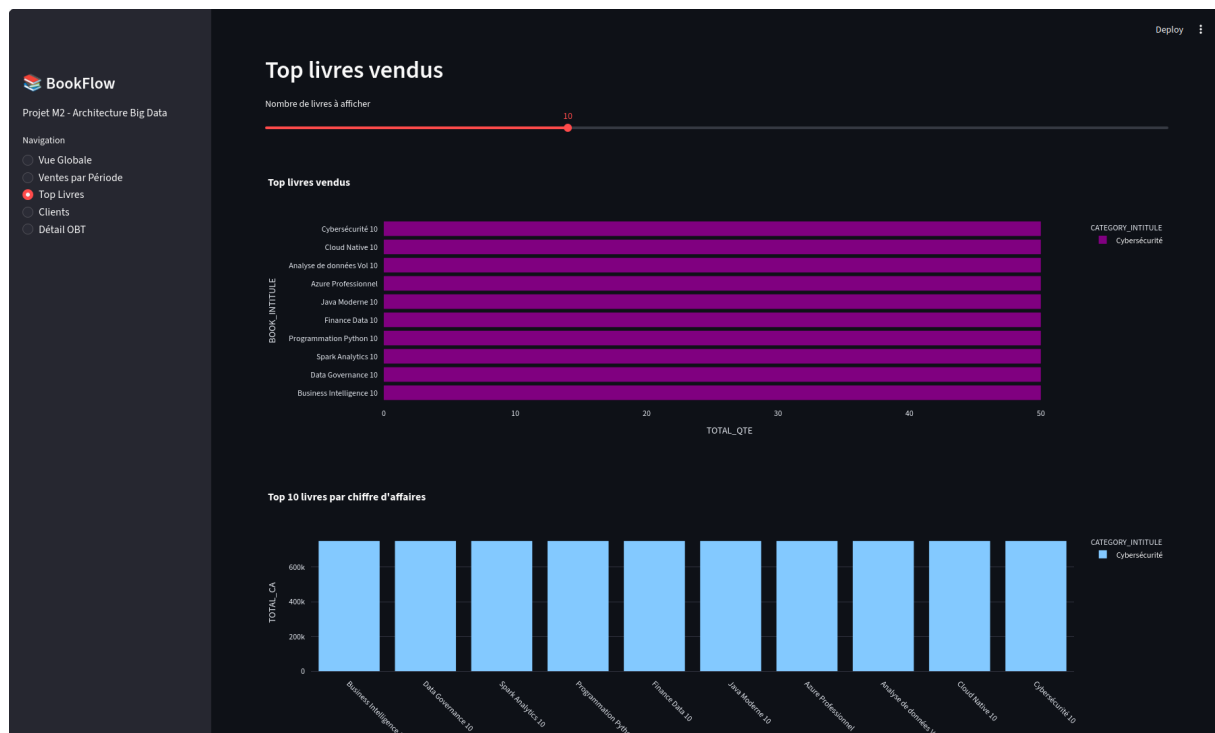


FIGURE 4.18 – Dashboard montrant les Top livres vendus



FIGURE 4.19 – Dashboard montrant les meilleurs clients

Conclusion

Le projet BOOKSHOP a permis de concevoir et de mettre en œuvre une chaîne complète de traitement et de valorisation des données, s’inscrivant dans les standards actuels du Data Engineering et de la Business Intelligence. À travers ce travail, il a été démontré qu’il est possible d’intégrer plusieurs technologies complémentaires au sein d’une architecture cohérente, performante et orientée décisionnel.

La solution développée repose sur une base de données transactionnelle PostgreSQL servant de source opérationnelle, un entrepôt de données cloud Snowflake destiné au stockage analytique, l’outil dbt pour les transformations SQL industrialisées, Apache Airflow pour l’orchestration automatisée du pipeline, ainsi que Streamlit pour la restitution visuelle des indicateurs clés. Cette combinaison technologique a permis de couvrir l’ensemble du cycle de vie de la donnée, depuis son extraction jusqu’à sa visualisation finale.

Les différentes étapes du pipeline ont été structurées selon une logique ELT moderne : ingestion des données vers la couche RAW, préparation dans STAGGING, modélisation décisionnelle dans WAREHOUSE, puis création d’un MART final sous forme de table consolidée OBT_SALES. Cette organisation en couches favorise la traçabilité, la qualité des données, la maintenabilité du code et la performance analytique.

Au-delà de la dimension technique, ce projet a également permis de renforcer des compétences pratiques en modélisation de données, conteneurisation avec Docker, automatisation des workflows, déploiement cloud et conception de tableaux de bord interactifs. Il constitue ainsi une application concrète des connaissances acquises en Big Data et en ingénierie des données.

En perspective, plusieurs améliorations pourraient être envisagées, notamment l’intégration de flux de données temps réel, l’ajout de tests de qualité automatisés, la mise en place d’alertes intelligentes ou encore le déploiement sur des infrastructures cloud plus avancées.

En définitive, le projet BookFlow Analytics Platform illustre l’importance stratégique de la donnée dans la prise de décision moderne et met en évidence la capacité des architectures analytiques contemporaines à transformer des données brutes en informations à forte valeur ajoutée .

BIBLIOGRAPHIE

- Apache Software Foundation (2024). Apache airflow documentation. Consulted for workflow orchestration concepts.
- Damji, J. S., Wenig, B., and Das, T. (2020). *Learning Spark : Lightning-Fast Data Analytics*. O'Reilly Media.
- dbt Labs (2024). dbt documentation. Consulted for data transformation modeling.
- Kimball, R. and Ross, M. (2013). *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*. Wiley.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Streamlit Inc. (2024). Streamlit documentation. Consulted for dashboard development.